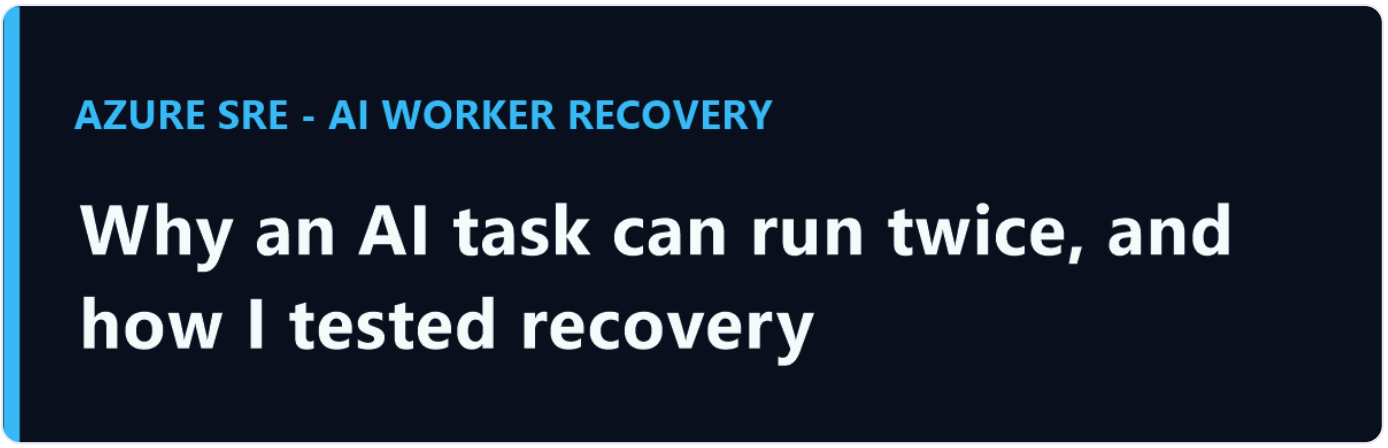
PUBLICATION REVIEW PACKET · 28 OCTOBER 2026

Why an AI Task Can Run Twice

Azure AI Ops, post 1 — article, evidence and release state

DRAFT: TRUE | NOT COMMITTED | NOT DEPLOYED | CONTENT VERIFIED



This packet summarises the Azure AI Ops post 1 package currently sitting in gcp-architect-blog, uncommitted. It lets you review the article, its evidence chain and the deployment implications without opening the source tree. The package is complete and internally consistent. Nothing has been committed, pushed or deployed.

1,233	47 / 3	6 / 6	0
words in the article (~6 minute read)	tests passed / skipped (50 collected, 0 failed)	manifest artifact hashes verified, plus 76/76 archive files	broken links or anchors in the rendered build

VERDICT

Ready for your review. Content and evidence pass every check performed. Release is blocked only on your approval, a four-item editorial decision list, and the mechanical publishing chores on page 7.

What you are reviewing

The article	A 1,233-word MDX post on duplicate AI task execution, with two reproducible failure windows and an explicit claim boundary.
The evidence	A public pytest transcript, a hash manifest, four code excerpts and a downloadable source ZIP whose 76 internal files are hash-verified.
The release state	Draft flag, uncommitted files, and a local git branch that has diverged from origin/main.

How to read this packet

Pages 2–3
The argument, the two failure windows and the full asset inventory.

Pages 4–5
Desktop and mobile previews, then the evidence chain and checks.

Pages 6–7
Git divergence, your review checklist and the release path.

Page 8
Absolute paths, provenance and reproduction commands.

The article in brief

Title	Why an AI Task Can Run Twice—and How I Tested Recovery
Repository path	`src/content/posts/sre-reliability/005-why-an-ai-task-can-run-twice.mdx`
Classification	Category sre-reliability · cloud azure · series azure-ai-ops #1 · proof tier 2
Dates	Publish date 2026-10-25 · evidence 25 Oct 2026 · packet 28 Oct 2026
Tags	azure · python · idempotency · queues · reliability · outbox · at-least-once

The argument

An AI request can finish successfully and still trigger a second model call. The first worker gets the answer, fails before saving it, and leaves an unfinished task behind. When another worker picks up that task, there is no durable result to reuse. With a paid provider, the retry can also incur a second charge.

The article tests two failure windows in a local reference implementation. The difference between them is one database commit: **a stored result lets the worker skip the model; a provider response that was never stored does not.**

Two failures, two outcomes

FAILURE POINT	DURABLE STATE	OBSERVED OUTCOME IN THE TEST
Result committed; queue acknowledgement fails	Task completed; result exists	Redelivery is acknowledged without another model call (ai.calls == 1)
Model returns; result commit fails	Unfinished claim; no result	After claim expiry, redelivery invokes the model again (ai.calls == 2)

Four duplicates, four different guards

DUPLICATE	GUARD IN THIS DESIGN	REMAINING LIMIT
Client retries a submission	Tenant-scoped idempotency key, payload hash, database uniqueness	Does not make worker execution happen once
Notification returns after completion	Terminal-status check	Requires a committed completion record
Workers compete for unfinished work	Row lock, expiring claim, per-attempt claim token	Cannot undo an external call already started
Provider receives a repeated request	Provider-supported deduplication, where documented	Depends on that provider's contract and retention window

The ownership fence, in one predicate

```
-- Repository.finish: a stale owner cannot overwrite the new owner
WHERE id=$1 AND claim_token=$2 AND status='running'
AND claim_until>clock_timestamp() RETURNING *
```

> **WHAT THE POST DOES NOT CLAIM**
>
> No Azure deployment, no live recovery, no paid model inference or billing measurement, and no provider-side deduplication. The queue is a policy double that does not reproduce real broker timing. Broader lab defects are disclosed in the ZIP's PUBLICATION-NOTES.md.

Asset inventory

All paths relative to C:\Users\koate\development\gcp-architect-blog. The article body is unchanged from the reviewed source; the only content delta this session was one ogImage frontmatter line.

GROUP	PATH	BYTES	ROLE
Article	src/content/posts/sre-reliability/005-why-an-ai-task-can-run-twice.mdx	10,302	Draft post
Evidence	public/evidence/why-an-ai-task-can-run-twice/azure-ai-ops-lab-source.zip	74,997	Downloadable source
	…/manifest.json	2,376	Hashes + claim map
	…/test-results.txt	5,153	Public transcript
	…/src/common/repository.py	8,850	Ownership fence
	…/src/worker/worker.py	6,792	Worker loop
	…/tests/test_recovery.py	5,038	The two tests
	…/tests/fakes.py	1,210	Queue/model doubles
Social	public/og/005-why-an-ai-task-can-run-twice.png	31,152	1200×630 card
	public/og/005-why-an-ai-task-can-run-twice-devto.png	30,134	1200×504 cover
Docs	docs/azure-ai-ops/HANDOVER.md	12,709	Continuation note
	docs/azure-ai-ops/SERIES_PLAN.md	10,199	Three-post plan
	docs/azure-ai-ops/POST_1_PUBLICATION_REVIEW.md	1,509	Preparation note
Review	docs/azure-ai-ops/review/post-1-desktop.png	468,372	1450px preview
	docs/azure-ai-ops/review/post-1-mobile.png	384,629	390px preview
	docs/azure-ai-ops/review/publication-tests.xml	6,172	JUnit source
Example	examples/why-an-ai-task-can-run-twice/README.md	1,400	Download note

What the ZIP contains

src/
API, worker, repository, Azure and local model clients, storage-queue adapter, settings and telemetry.

tests/
50 collected tests across recovery, boundaries, contracts, integration and tracing.

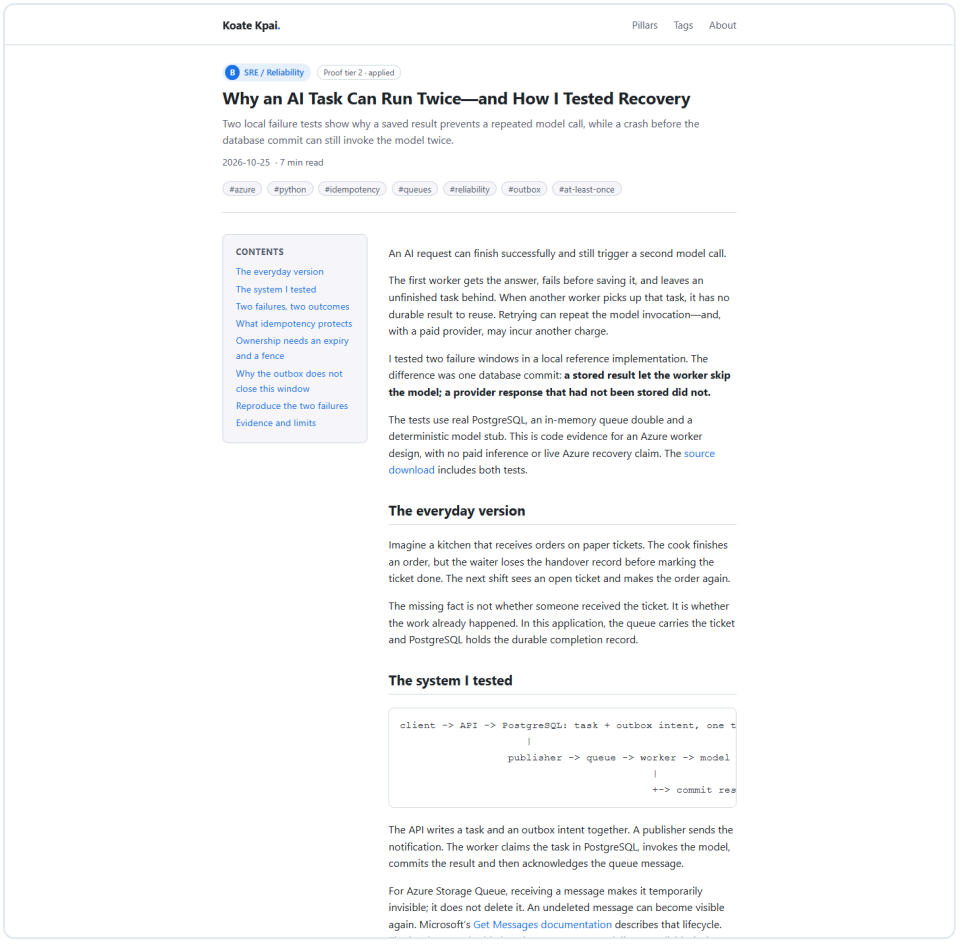
infra/ + migrations/
Bicep modules, Alembic migration, KQL queries, runbook and validation notes.

Also includes PUBLICATION-NOTES.md, MANIFEST.sha256, README, Dockerfile and dependency locks.

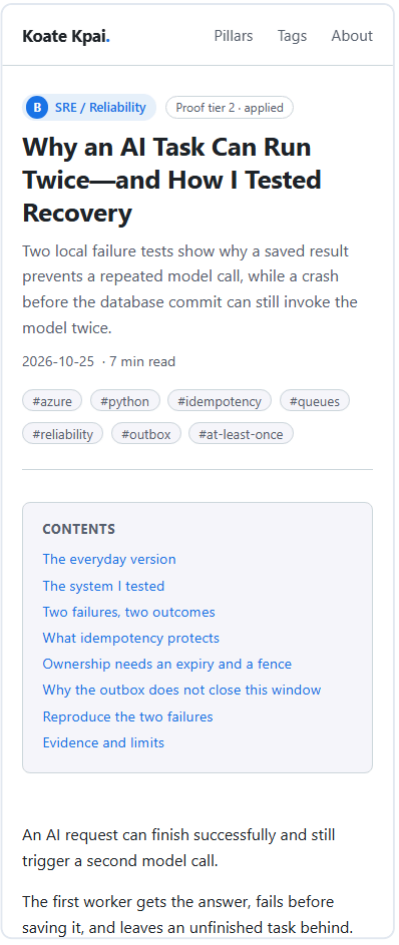
> **THE EVIDENCE BUNDLE IS FROZEN**
> The directory must not gain files. Re-running the evidence finalizer rewrites manifest.json and hashes every file present, so a stray cache or supplementary log would silently change the published record.

The rendered article

Captured on 25 October 2026 from the local build at 1440px and 390px. The page had no horizontal overflow and every in-page anchor resolved.



Desktop, 1440px — top of article



Mobile, 390px

WHAT TO LOOK FOR

The opening paragraph states the failure in plain language before any queue terminology. The two-window table and the idempotency table carry the argument. The code block is the actual archive test, not a reconstruction. The closing section names what the local run cannot prove.

Social card, 1200×504



The wider 2.38:1 variant used for Dev.to and Hashnode, where a 1200×630 card would be cropped. Both cards are generated reproducibly by scripts/gen-og.ps1.

Evidence chain and verification

1. CLAIM
Each assertion in the article names the test that supports it.

2. TEST
The named test exists in the archive and in the public excerpt.

3. TRANSCRIPT
The public run shows the test passing, with the expected call count.

The crux, quoted from the archive

```
# test_provider_response_before_commit_can_repeat (excerpt)
repo.finish = crash
with pytest.raises(OSError):
    await worker.one()
...
assert ai.calls == 2  # Deliberate exactly-once limitation, not hidden.
assert (await repo.get("tenant-a", task["id"]))["status"] == "completed"
```

The excerpt is verbatim from tests/test_recovery.py lines 117–138; ellipses mark omitted lines.

Checks performed

CHECK	RESULT
Article versus reviewed output	✓ Identical except the added ogImage line
Manifest artifact hashes	✓ 6 of 6 SHA-256 and byte counts match
Source ZIP internal manifest	✓ 76 of 76 files match; 0 missing; 0 mismatched
Test transcript	✓ 50 collected: 47 passed, 3 skipped, 0 failed
Both article tests	✓ Present and passing; call counts 1 and 2 as described
Ownership fence SQL	✓ Matches Repository.finish in the archive
Build with draft enabled	✓ 66 pages; 7 Mermaid diagrams rendered
Links and anchors	✓ 19 internal + 3 evidence links resolve; 0 broken anchors
Evidence files in build output	✓ ZIP, manifest and transcript copied to dist/
OG images	✓ Both cards render with the correct title and footer

Key hashes

ARTIFACT	SHA-256 (FIRST 16 HEX)
azure-ai-ops-lab-source.zip	228f01473d8683ea
manifest.json	21d8eb2dc2dd6114
test-results.txt	23cf3b95ca81b8be
src/common/repository.py	7c45db21e77e13fc
src/worker/worker.py	4d042794c88b2487

WHY THREE TESTS ARE SKIPPED

Two Azurite integration tests require an emulator that was not configured, and the POSIX SIGTERM process test cannot run on Windows. All three are environmental skips, not failures. The suite ran on Python 3.14.5 with PostgreSQL 18.4 on a fresh disposable database.

Current git and deployment state

Branch	`main`
Local HEAD	`d3d010c — docs: record the completed live run and post-run handover`
Remote HEAD	`1e29591 — Add Lambda results graphic and establish post visual standard`
Common ancestor	`85e06e2 — post: publish 004 SLOs for Cloud Run`
Divergence	Local is **17 commits ahead** and **4 commits behind** origin/main
Azure assets	All untracked; none appear in git ls-files or origin/main
Deployment	Not run. A push to main triggers the Firebase deploy workflow.

```
85e06e2  common ancestor
   |-- local main   : 17 commits (AWS Lambda work)  -> d3d010c
   '-- origin/main  :  4 commits (published AWS post) -> 1e29591
```

DO NOT PUSH THIS BRANCH AS-IS

The histories have diverged. A normal push is rejected as non-fast-forward; a force-push would delete the four published commits on origin/main, including the live AWS Lambda case study. Publishing must branch from origin/main and carry only the Azure files.

Review checklist

1. Read the eight article sections: everyday version, system, two failures, idempotency, ownership, outbox, reproduction, evidence and limits.
2. Confirm the two-window table matches your understanding: one commit decides whether the model is called twice.
3. Open manifest.json and verify the four claims keys point at real tests.
4. Open test-results.txt and find both PASSED lines and the 47/3 summary.
5. Decide the source question: downloadable ZIP (current) versus a codeRepo URL (series-plan gate).
6. Confirm the release date: frontmatter is 2026-10-25; release would be 2026-10-28.

WHAT IS ALREADY DONE

Article staged · evidence hashed and frozen · social cards generated · previews captured · internal docs and handover written · build and link checks passed. Nothing depends on a further content edit before release, unless you change a decision above.

WHERE THE DIVERGENCE CAME FROM

All 17 local-only commits are the AWS Lambda implementation line. origin/main already contains a published version of that work in four different commits. The local line is therefore superseded for publication purposes: keep it for reference, but do not use it as the base for the Azure release.

Release path and open decisions

If you approve

1. Set draft: false and publishDate: 2026-10-28 in the article frontmatter.
2. Move backlog topic 022 to published, with publishedAs, date and a corrected note (47 passed, 3 skipped).
3. Run npm run score and review the regenerated scorecard diff.
4. Update the publication review note and the series-plan gate, recording the ZIP decision honestly.
5. Build with the draft enabled; verify the article page, evidence ZIP and OG image in dist/.
6. Branch from origin/main, add only the Azure paths, commit, and push the branch.
7. Open a PR for a Firebase preview-channel deploy; merge to main for production.

Open decisions

DECISION	CONTEXT	RECOMMENDATION
Source link	The post links a source ZIP. The series plan's gate says a repository link should exist, but the source was delivered as an archive with no commit.	Accept the ZIP; do not invent a repository or commit.
Publish date	Frontmatter says 2026-10-25; the release would be 2026-10-28. The body cites the 25 October evidence run.	Set the frontmatter to the release date; keep the evidence sentence.
OG delta	The reviewed MDX gained one ogImage line this session.	Accept; social image only, body unchanged.
Posts 2 and 3	006 and 007 remain drafts with live Azure evidence pending.	Keep out of this release.

Publish readiness

DONE	OPEN
✓ Article content staged	○ Your review and approval
✓ Evidence frozen and hashed	○ Flip draft and date
✓ Both failure tests verified	○ Backlog and scorecard
✓ Build and links checked	○ Commit on a branch from origin
✓ Social cards and previews	○ PR preview, then merge
✓ Docs and handover written	○ Post-deploy URL checks

RELEASE IS DELIBERATELY PAUSED HERE

The article remains invisible to the build, RSS, sitemap and category pages while draft: true is set. Current dist/ was rebuilt with the draft flag on (59 pages) and does not contain the article page.

Appendix — where everything lives

Repository

```
C:\Users\koate\development\gcp-architect-blog
  src/content/posts/sre-reliability/005-why-an-ai-task-can-run-twice.mdx
  public/evidence/why-an-ai-task-can-run-twice/      (7 files)
  public/og/005-why-an-ai-task-can-run-twice*.png
  docs/azure-ai-ops/                                 (plan, review, handover)
  docs/azure-ai-ops/review/                          (previews, JUnit XML)
  examples/why-an-ai-task-can-run-twice/README.md
```

Reviewed originals and tooling

```
C:\Users\koate\Documents\Codex\2026-10-25\lo\outputs
  Why-an-AI-Task-Can-Run-Twice.md
  AI-worker-test-results.txt
  AI-worker-post-preview.png / AI-worker-post-mobile.png
  Publication-review.md

C:\Users\koate\Documents\Codex\2026-10-25\lo\work
  prepare_post.py / finalize_evidence.py / check_preview.mjs
  publication-tests.xml / test-results.txt / build-results.txt
```

Original runnable source (23 October archive)

```
C:\Users\koate\Documents\Codex\2026-10-23\i-w\outputs\
  Azure-AI-Ops-Lab-Revised-Source\azure-ai-ops-lab
```

Reproduce the two article tests

```
python -m pytest -v `
  tests/test_recovery.py::test_post_commit_ack_failure_skips_second_ai `
  tests/test_recovery.py::test_provider_response_before_commit_can_repeat
```

Requires Python 3.14 and a disposable PostgreSQL database whose name ends in _test. Set TEST_DATABASE_URL and MIGRATION_DATABASE_URL, then run python -m alembic upgrade head.

Useful facts for review

Article size	10,302 bytes · 213 lines · 1,233 words
Evidence directory	7 files · 104,416 bytes
Source archive	78 ZIP entries · 76 manifest-verified files + manifest + publication note
Test environment	Python 3.14.5 · PostgreSQL 18.4 · Windows
Run identifier	`20261025-local-publication`
Recorded at (UTC)	`2026-10-25T13:31:34Z`

ONE-LINE SUMMARY

A complete, locally verified Azure AI Ops article and evidence package, held as a draft and uncommitted, ready for a human review of its claims, its ZIP-based source, and its release date before it is published.